

# mol2db2 C++ - Complete Setup Guide

## Step 1: Create Directory Structure

```
bash
# Create main project directory
mkdir -p mol2db2-cpp
cd mol2db2-cpp

# Create all subdirectories
mkdir -p src/core
mkdir -p src/algorithms
mkdir -p src/molecular
mkdir -p src/hierarchy
mkdir -p src/main
mkdir -p tests
mkdir -p build
mkdir -p bin
```

## Step 2: Download All Files

I'll provide a complete file manifest. Save each file to its designated location:

### Core Files (src/core/)

- geometry.h
- geometry.cpp
- combinatorics.h
- unionfind.h
- unionfind.cpp
- priodict.h

### Algorithm Files (src/algorithms/)

- floydwarshall.h
- floydwarshall.cpp
- shortestpaths.h
- shortestpaths.cpp
- buckets.h
- buckets.cpp
- pca.h
- pca.cpp
- divisive\_clustering.h
- divisive\_clustering.cpp

### Molecular Files (src/molecular/)

- mol2.h
- mol2.cpp
- clash.h
- clash.cpp
- hydrogens.h
- hydrogens.cpp

### Hierarchy Files (src/hierarchy/)

- hierarchy.h
- hierarchy.cpp

#### Main Program (src/main/)

- mol2db2.cpp

#### Build Files (root directory)

- Makefile

#### Documentation (root directory)

- README.md

#### Test Files (tests/)

- test\_geometry.cpp
- test\_unionfind.cpp
- test\_mol2.cpp

### Step 3: Install Dependencies

#### On Ubuntu/Debian:

```
bash
sudo apt-get update
sudo apt-get install -y build-essential libeigen3-dev zlib1g-dev
```

#### On macOS:

```
bash
brew install eigen zlib
```

#### On CentOS/RHEL:

```
bash
sudo yum install -y gcc-c++ eigen3-devel zlib-devel
```

#### Verify Eigen3 Installation:

```
bash
# Find Eigen include directory
find /usr -name "Eigen" -type d 2>/dev/null | head -5

# Common locations:
# /usr/include/eigen3/Eigen
# /usr/local/include/eigen3/Eigen
# /opt/homebrew/include/eigen3/Eigen (macOS ARM)
```

If Eigen is in a non-standard location, update the Makefile:

```
makefile
INCLUDES = -I/your/eigen/path/eigen3
```

### Step 4: Build the Project

```
bash
# From mol2db2-cpp directory
make clean
make

# You should see:
# - Compiling messages for each .cpp file
# - Final: "Build complete: bin/mol2db2"
```

### Expected Output:

```
g++ -std=c++17 -O3 -Wall -Wextra ... -c src/core/geometry.cpp -o build/geometry.o
g++ -std=c++17 -O3 -Wall -Wextra ... -c src/core/unionfind.cpp -o build/unionfind.o
...
Build complete: bin/mol2db2
```

### Common Build Issues:

#### Issue 1: "Eigen/Dense: No such file"

```
bash
# Find Eigen
locate Eigen/Dense

# Update Makefile INCLUDES line to correct path
```

#### Issue 2: "cannot find -lz"

```
bash
# Install zlib
sudo apt-get install zlib1g-dev
```

#### Issue 3: C++17 not supported

```
bash
# Check GCC version
g++ --version

# Need GCC 7.0+, upgrade if necessary:
sudo apt-get install g++-9
export CXX=g++-9
make clean && make
```

### Step 5: Create Test Data

Let me create a minimal test case to verify everything works:

#### Create test\_data directory:

```
bash
mkdir test_data
cd test_data
```

#### Create minimal MOL2 file (test.mol2):

@<TRIPOS>MOLECULE

BENZENE

12 12 0 0 0

SMALL

USER\_CHARGES

mmff94s\_NoEstat = 0.00

@<TRIPOS>ATOM

|    |    |         |         |        |      |   |     |         |
|----|----|---------|---------|--------|------|---|-----|---------|
| 1  | C1 | 0.0000  | 1.4000  | 0.0000 | C.ar | 1 | <0> | -0.1500 |
| 2  | C2 | 1.2124  | 0.7000  | 0.0000 | C.ar | 1 | <0> | -0.1500 |
| 3  | C3 | 1.2124  | -0.7000 | 0.0000 | C.ar | 1 | <0> | -0.1500 |
| 4  | C4 | 0.0000  | -1.4000 | 0.0000 | C.ar | 1 | <0> | -0.1500 |
| 5  | C5 | -1.2124 | -0.7000 | 0.0000 | C.ar | 1 | <0> | -0.1500 |
| 6  | C6 | -1.2124 | 0.7000  | 0.0000 | C.ar | 1 | <0> | -0.1500 |
| 7  | H1 | 0.0000  | 2.5000  | 0.0000 | H    | 1 | <0> | 0.1500  |
| 8  | H2 | 2.1651  | 1.2500  | 0.0000 | H    | 1 | <0> | 0.1500  |
| 9  | H3 | 2.1651  | -1.2500 | 0.0000 | H    | 1 | <0> | 0.1500  |
| 10 | H4 | 0.0000  | -2.5000 | 0.0000 | H    | 1 | <0> | 0.1500  |
| 11 | H5 | -2.1651 | -1.2500 | 0.0000 | H    | 1 | <0> | 0.1500  |
| 12 | H6 | -2.1651 | 1.2500  | 0.0000 | H    | 1 | <0> | 0.1500  |

@<TRIPOS>BOND

|    |   |    |    |
|----|---|----|----|
| 1  | 1 | 2  | ar |
| 2  | 2 | 3  | ar |
| 3  | 3 | 4  | ar |
| 4  | 4 | 5  | ar |
| 5  | 5 | 6  | ar |
| 6  | 6 | 1  | ar |
| 7  | 1 | 7  | 1  |
| 8  | 2 | 8  | 1  |
| 9  | 3 | 9  | 1  |
| 10 | 4 | 10 | 1  |
| 11 | 5 | 11 | 1  |
| 12 | 6 | 12 | 1  |

#### Create SOLV file (test.solv):

BENZENE 12 0.0000 -5.234 285.432 -0.234 -5.468  
-0.1500 -0.234 28.543 -0.023 -0.257  
-0.1500 -0.234 28.543 -0.023 -0.257  
-0.1500 -0.234 28.543 -0.023 -0.257  
-0.1500 -0.234 28.543 -0.023 -0.257  
-0.1500 -0.234 28.543 -0.023 -0.257  
-0.1500 -0.234 28.543 -0.023 -0.257  
0.1500 -0.123 18.234 -0.012 -0.135  
0.1500 -0.123 18.234 -0.012 -0.135  
0.1500 -0.123 18.234 -0.012 -0.135  
0.1500 -0.123 18.234 -0.012 -0.135  
0.1500 -0.123 18.234 -0.012 -0.135  
0.1500 -0.123 18.234 -0.012 -0.135

#### Create name file (name.txt):

name.txt dummy BENZENE c1ccccc1 dummy Benzene\_Test\_Molecule

#### Step 6: Run First Test

```
bash
cd .. # back to mol2db2-cpp directory

# Run with verbose output
./bin/mol2db2 \
-m test_data/test.mol2 \
-s test_data/test.solv \
-n test_data/name.txt \
-o test_data/output.db2.gz \
-v
```

### Expected Output:

```
mol2db2 - Converting MOL2 to DB2 format
Input MOL2: test_data/test.mol2
Input SOLV: test_data/test.solv
Output DB2: test_data/output.db2.gz
Reading MOL2 file...
Converting atom types...
Reading solvation data...
Processing terminal hydrogens...
Found 6 rotatable hydrogens
Rotating hydrogens...
Total conformations: 64
Building hierarchy...
Counting unique positions...
Unique positions per atom: 12
Finding rigid component...
Rigid atoms: 6
Finding conformations...
Total conformations: 2
Identifying clashes...
Clashed sets: 0
Making clouds...
Writing DB2 file...
test_data/output.db2.gz written
Conversion complete!
```

### Step 7: Verify Output

```
bash
# Check output file was created
ls -lh test_data/output.db2.gz

# Should show a .gz file of reasonable size (few KB for this small test)

# Verify it's gzipped
file test_data/output.db2.gz
# Output: test_data/output.db2.gz: gzip compressed data

# Peek at contents (first few lines)
zcat test_data/output.db2.gz | head -20
```

### Expected DB2 Format:

```
M      BENZENE   none 12 12   ...
M +0.0000  -5.234  -0.234  -5.468  285.432
M c1ccccc1
M Benzene_Test_Molecule
M +999.9990
A 1 C1  C.ar  1 3 -0.1500  -0.234  -0.023  -0.257  28.543
...
```

### Step 8: Run Unit Tests

```
bash
make test
```

### Expected Output:

```
Running tests...
✓ test_distL2Squared3 passed
✓ test_cross passed
...
All tests passed!
```

## Step 9: Performance Test (Optional)

Create a larger test with multiple conformations:

```
bash
# This would normally come from your molecular modeling workflow
# For now, we'll use the simple test
time ./bin/mol2db2 -m test_data/test.mol2 -s test_data/test.solv -o test_data/output.db2.gz
```

## Troubleshooting

### Program crashes immediately:

```
bash
# Run with debugger
gdb ./bin/mol2db2
(gdb) run -m test_data/test.mol2 -s test_data/test.solv -o test_data/output.db2.gz -v
(gdb) bt # backtrace when it crashes
```

### "Hierarchy too large" error:

```
bash
# Reduce limits
./bin/mol2db2 \
-m test_data/test.mol2 \
-s test_data/test.solv \
-o test_data/output.db2.gz \
--limitconf 10000 \
--limitset 50000 \
-v
```

### Valgrind memory check:

```
bash
valgrind --leak-check=full --show-leak-kinds=all \
./bin/mol2db2 \
-m test_data/test.mol2 \
-s test_data/test.solv \
-o test_data/output.db2.gz
```

## Step 10: Process Real Data

Once tests pass, process your actual data:

```
bash
./bin/mol2db2 \
-m /path/to/your/ligand.mol2.gz \
-s /path/to/your/ligand.solv \
-n /path/to/your/name.txt \
-o /path/to/output.db2.gz \
-v \
--time
```

## Success Checklist

- ☐ All dependencies installed (Eigen3, zlib)
- ☐ Project compiles without errors
- ☐ Test data created
- ☐ Program runs on test data
- ☐ Output file created and valid
- ☐ Unit tests pass
- ☐ Ready for real data

## Next Steps

1. **Validate against Python output** (if you have it)
2. **Profile performance** on real datasets
3. **Add custom parameter files** if needed
4. **Integrate into your pipeline**

## Getting Help

If you encounter issues:

1. **Check build output** for specific errors
2. **Verify dependencies** are installed correctly
3. **Test with minimal data** first
4. **Check file permissions** for input/output
5. **Review error messages** carefully

Would you like me to create any of these test files for you, or help debug a specific issue?